A struct for one account snapshot

Date: 15 October 2026. Integrated: 17 October 2026. Task: L001c, one position and its cash. Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a small C++ struct keep cash and one share count together while preserving the earlier marked-wealth calculation? Prediction before implementation: cash 800, four shares and a mark of 50 still give holdings 200 and wealth 1000. Grouping values changes their representation, not the accounting equation. The struct alone will not enforce financial validity.

L001a introduced cash plus marked holdings. L001b derived one buy and a fee. Today introduces only a struct, its fields and initialization. No trade is executed. This is a synthetic fixed snapshot with no historical period, market observations, strategy, fitting or parameter search. The protocol is a hand calculation, warning-clean C++17 build, independent Decimal reference, two exact runs, repeatable JSON, and a deliberately swapped-field negative control.

Beginner explanation and vocabulary

A type describes what a value contains. An object is one concrete value of that type. A struct lets us name related fields together. AccountState is the type; initial is one object. Its two fields are cash_units and shares. Together they describe one account holding one known asset. The name deliberately refers to an account: cash is not a share position in that asset.

Field or input	Meaning	Units / initial value
cash_units	Cash held by this account	Whole money units: 800
shares	Quantity of the sole asset	Whole shares: 4
mark_units_per_share	External valuation price	Money units per share: 50

The mark is outside the struct because it is a valuation input, not cash or quantity owned. Holdings value and wealth are calculated rather than stored as extra fields that could become inconsistent with the underlying quantities. There is one implicit asset and one money unit; there are no identifiers, currencies or multiple positions to resolve in this lesson.

Mathematics and worked example

H = q * p; W = C + H. Here C is cash, q is shares, p is the mark, H is holdings value and W is total marked wealth. Shares times money per share gives money, which can be added to cash. For the initial object, H = 4*50 = 200 and W = 800+200 = 1000. For the separate empty object, C = 0 and q = 0, so H = 0 and W = 0 at the same mark.

These are two independent snapshots, not a sequence in which the first account loses its money. The empty object checks initialization; it is not a deposit, sale, closing trade or bankruptcy. All calculated integers are between 0 and 1000 in the correct fixture, safely inside int's minimum required range. This is a bounded argument, not arbitrary-input overflow protection.

The C++ example, line by line

The source is cpp/projects/portfolio_accounting/account_state.cpp. The first line includes iostream for printed output. The comment describes the intended scope. The declaration starts with `struct AccountState {` and introduces a type with two named members.

`int cash_units;` declares the whole-money field. `int shares;` declares the whole-share field. The closing `};` ends the type definition; the semicolon is required. Defining the type does not itself create an account object. Both members are public, so code can access them directly.

Inside main, `const AccountState initial{800, 4};` creates one initialized object. The first number initializes cash_units and the second initializes shares, in declaration order. The const qualifier prevents subsequent modification of this particular object. It does not make every AccountState object constant or validate the values used to initialize one.

`const int mark_units_per_share = 50;` defines the external mark. The dot in `initial.shares` selects that object's shares member. holdings_units multiplies this quantity by the mark; wealth_units adds initial.cash_units. The first output statement prints all five quantities. The stream operator << prints the following value; '\n' ends the line.

`const AccountState empty{};` creates a separate object. For this specific simple aggregate with two int fields and no custom initialization, empty braces initialize both fields to zero. The final output statement prints its cash, shares and wealth. main then ends successfully. There are no constructors, inheritance, headers, setters or buy functions to learn today.

Why initialization order matters

AccountState initial{4, 800} is valid C++: both fields are int. It means cash 4 and 800 shares, not cash 800 and four shares. At mark 50, that mistaken snapshot has wealth 40004. A compiler cannot infer that the author intended different financial units when both values fit int.

The deliberate scratch mutation swaps exactly these initializer values. It compiles without warnings but the verifier rejects its output. We compare the individual fields as well as wealth, because compensating field errors can sometimes leave a total unchanged. No broken source replaces the correct example. The control tests the verifier, not market behaviour.

Use explicit initialization. A local mutable declaration such as AccountState state; without an initializer does not initialize these int members to known amounts. Do not read them before assignment. We do not execute this unsafe counterexample or treat a chance output as evidence. Similarly, a struct with public int fields can represent negative cash or shares: today's representation has no rejection policy. That belongs in the next validated-operation lesson.

Reproduce, checks and observed results

From the repository root, run: `.venv/bin/python scripts/verify_account_state.py`. It freshly compiles the source into a temporary directory using c++ with -std=c++17, -Wall, -Wextra, -Wpedantic and -Werror, then removes that temporary build on exit. The hand-authored account_state_expected.txt specifies both output lines before execution.

The Python reference values holdings by adding Decimal(50) four times, independently of the C++ multiplication. Hand-known totals and the golden output also constrain the reference. Both executable runs match every output byte, empty stderr and exit status 0. Two complete verifier invocations, each with a fresh build, produce identical result JSON. Observed initial cash is 800, shares 4, holdings 200 and wealth 1000; the empty object's values are all zero.

The swapped-initializer control compiles but exits the verifier with status 1 and the intended reference-mismatch diagnostic. An actual Python -O invocation is refused by an explicit guard, so disabled assertions cannot produce a false pass. The new example passed its first run. Apple Clang 21 was used; these checks make no claim about every compiler or platform.

Full make check passes all 608 Python tests, lint, formatting and strict package typing, plus the existing C++ savings/parser, boundary, fractional-failure, readiness, marked-wealth and buy-equation checks. The integrated verifier also passes lint and formatting. Dependency consistency passes with a nonfatal disabled-cache warning. No dependency was added, no test-coverage percentage measured and no empirical return or speed improvement claimed.

Costs, assumptions, failure cases and limits

No buy occurs, so there is no transaction fee to subtract today. Yesterday's fee of 3 remains part of its separate buy example; placing cash and shares in a struct does not reverse it. If a future operation buys shares, it must account for price, quantity and the fee together. A struct does not establish affordability, atomic rejection or correct settlement.

Only whole arbitrary money units and whole shares are used. Fractional money, rounding, large values, borrowing, short sales, dividends, taxes and settlement delays need explicit future policies. int has finite range, and neither grouping fields nor using const prevents arithmetic overflow for arbitrary inputs. Negative or unaffordable orders are not accepted external inputs here; there is no external interface. Their validation and unchanged-state tests remain L001d/e.

The mark is assumed known. No decision time, execution delay, future-price invariance or real market fill is modelled. Q004a/Q005b continue to gate empirical data; the agreement of two implementations establishes this fixture's mechanics, not profitability or market realism.

Interview explanation

I separate an account's owned quantities from its valuation price. A simple struct holds cash and shares; wealth is derived from them. Brace initialization follows member order, so I test each field against a reference. The struct expresses state but does not enforce financial invariants. A validated buy operation and its rejection behaviour are separate work.

Primary sources consulted on October 15

Microsoft, struct (C++): user-defined composite types, member access and public-by-default members. The two financial fields and the accounting interpretation are our design choices. https://learn.microsoft.com/en-us/cpp/cpp/struct-cpp?view=msvc-170

Microsoft, Initializers: aggregate member initialization in order, empty initialization and the distinction from uninitialized local scalar members. The claims above concern only our simple two-int aggregate; they are not universal claims about all class initialization. https://learn.microsoft.com/en-us/cpp/cpp/initializers?view=msvc-170

Python, subprocess: captured output, return codes and timeouts support checking compiled programs. The independent accounting calculation and golden fixture are authored here. https://docs.python.org/3/library/subprocess.html#subprocess.run

Progress, publication and next task

L001c was studied on October 15 with a zero commit draw and saved in ignored pending storage. It is integrated on October 17 after all 11 original input and nine artifact hashes matched. The original result is preserved. Paths and Make integration are updated; current fresh builds, negative controls, full checks and this revised PDF are revalidated before publication.

October 17's shared draw is one. This coherent integration uses the available slot; subsequent new study material stays pending. No October 16 lesson artifact was found, so no completion is inferred for that date. Commit/push outcomes are recorded in the run briefing and integration receipt. Personal review remains Not yet reviewed. No input is needed for synthetic continuation; lawful source evidence remains necessary before real-data acquisition.

Next is L001d: one validated buy function with invalid/unaffordable orders leaving state unchanged. L001e boundaries, L001f reproduction and L002 timing remain unfinished. C003b/C004/C005 and later strategy prerequisites remain; do not accelerate lessons to recover calendar dates. Continue at 08:00 Europe/London including daylight-saving changes through December 31, then request scope.

Check your understanding

1. How do AccountState and initial differ?

2. What does initial{800, 4} mean, and why is reversing the values dangerous?

3. Why are the mark and wealth not extra fields in this account snapshot?

4. Does const or a struct guarantee nonnegative amounts and affordable trades?

5. Does empty{} mean the initial account was liquidated?

Suggested answers

1. AccountState is the type with two named fields. initial is one object with cash 800 and four shares.

2. Values initialize members in order: cash first, shares second. Reversing them still compiles but gives cash 4 and 800 shares, worth 40004 at mark 50. Financial units are not checked by int.

3. The mark is a valuation input. Wealth is derived as cash plus shares times that mark; storing another total would create another value that must be kept consistent.

4. No. const prevents changing this object after initialization. The struct groups fields; it does not reject invalid starting values or implement an affordable, atomic buy operation.

5. No. It is a separate zero-initialized object. No trade or transfer connects these snapshots.